TP FINAL · 2DO CUATRIMESTRE 2026

TP Final — Infra para Ciencia de Datos

📦 Entrega: dom 15/11, 23:59 | 🎤 Presentación: jue 19/11 | 🐳 Airflow 3.1.5

1 Objetivo

Construir un pipeline de datos **end-to-end** sobre **una API pública**, con arquitectura medallion. Orquestarlo con **Airflow**, guardarlo en **PostgreSQL** y mostrar Gold en un **dashboard**. Que todo esté containerizado: con `docker compose up` el stack levanta y **empieza a generar datos solo**.

2 Cómo se evalúa

1. **Entrega y funcionamiento**
   Un `docker-compose.yml` que levante todo (la base, Airflow y el dashboard) y que con `docker compose up` el pipeline empiece a generar datos solo.
2. **Cumplir la consigna**
   Las versiones y tecnologías pedidas: **Airflow 3.1.5** (imagen `apache/airflow:3.1.5` — la 2 no sirve, el código de la 2 no corre en la 3), arquitectura **Medallion** (Bronze → Silver → Gold) y **SQL** en las transformaciones. Más el **dashboard en Streamlit** sobre las tablas Gold, y el **README del grupo**: API elegida, modelo de datos, cómo levantarlo, usuario y contraseña de Airflow, e integrantes.
3. **Trabajo colaborativo en git**
   Commits de varios integrantes, repartidos en el tiempo.
4. **Presentación final**
   Tiempos y formas: 7 a 10 minutos, dinámica y clara. Qué problema eligieron, cómo lo resolvieron y a qué resultado llegaron.

Adicional, para la nota individual:

5. **Participación durante la cursada**
   Las entregas de cada clase, en tu PR.

> ⚠️ **Y el requisito crítico: el stack tiene que arrancar a correr SOLO.**
> Cuando se haga `docker compose up`, el pipeline empieza a correr sin que haya que activar DAGs a mano ni crear schemas manualmente: **Bronze corre al levantar el stack y, cuando termina, dispara Silver y después Gold.**

3 Cómo se entrega

1. **Usen el número de grupo** que ya tienen asignado (`G01`, `G02` ...). Va en la rama, en la carpeta y en el título del PR.
2. Creen la rama `tpfinal/G<NN>` desde `main`, y la carpeta `TpFinal/grupos/G<NN>/`.

3. Abran un **PR** contra `main`, con título `TP Final - G<NN> - <API>`. En el cuerpo: integrantes (nombre + usuario de GitHub), API elegida e idea para Gold.
4. Trabajen en esa rama: **cada push actualiza el PR solo**.
5. **Entrega: domingo 15 de noviembre, 23:59** (hora Argentina). Vale **lo que esté en la rama** en ese momento: no hay que apretar ningún botón.

> El `.env` del grupo **sí va al repo**, a propósito: así el stack levanta con un solo comando. Son credenciales de juguete (`admin/admin`). Una API key real no va ahí: se pasa como variable del host y el `.env` la lee con `${MI_API_KEY}`.

4 La presentación final

Jueves 19 de noviembre de 2026 · **remota, por videollamada** · **7 a 10 minutos** por grupo, con el dashboard corriendo. Muestren la idea del proyecto y el pipeline funcionando. Después, una ronda corta de **preguntas**.

> **Formato libre, pero descargable:** la presentación se sube al campus, así que tiene que quedar en **PDF o PPT**. Apunten a unas **5 o 6 diapositivas**.

Qué mostrar

- **La API y la pregunta**: qué datos trae y qué pregunta de negocio responden.
- **Qué hicieron en cada capa**: Bronze → Silver → Gold.
- **Que arranca solo**: si pueden, en vivo — un `docker compose up` y cómo Bronze dispara Silver y Gold sin tocar nada.
- **El dashboard** y qué cuentan las visualizaciones.
- **Una decisión técnica** que hayan tomado, y por qué.
- Todo lo adicional que les parezca útil.

Tips

- Que sea **dinámico**.
- No hace falta que hablen todos, pero **todos tienen que estar listos para responder**.
- **Cámara prendida** durante la exposición.

5 Fuentes posibles

Elijan **una**. Todas se actualizan al menos cada hora. Pueden repetir API entre grupos, o proponer otra consultando con el docente.

> **No valen APIs de crypto ni de exchanges**: CoinGecko es el ejemplo que usamos en clase.

> 💡 Las ideas por capa son **orientativas**, no requisitos.

01 Open-Meteo — Clima

`open-meteo.com`

Sin auth, sin límite | Cada hora

BRONZE Clima actual + forecast 7 días para N ciudades: temperatura, lluvia, humedad, viento.

SILVER Normalizar unidades (°C, km/h), separar forecast de medición real, sensación térmica.

GOLD `fact_clima_diario`, `dim_ciudad`, `dim_tiempo`. Dashboard: comparación entre ciudades, alertas de temperatura extrema.

02 OpenWeatherMap — Clima

`api.openweathermap.org`

API key gratis | Cada 10 min

BRONZE Clima actual + pronóstico 5 días para ciudades configuradas.

SILVER Parsear JSON anidado, extraer condiciones (nublado, lluvia, despejado), UTC → local.

GOLD `fact_pronostico_vs_real`, `dim_condicion_climatica`. Dashboard: precisión del pronóstico, patrones por estación.

03 OpenAQ — Calidad del aire

`api.openaq.org/v2`

Sin auth | Cada hora

BRONZE Mediciones de PM2.5, PM10, CO, NO2, O3 por estación de monitoreo global.

SILVER Filtrar mediciones inválidas (negativos, outliers), estandarizar unidades (µg/m³), enriquecer con país/ciudad.

GOLD `fact_calidad_aire_diaria` (AQI calculado), `dim_estacion`, `dim_ciudad`. Dashboard: ranking de ciudades más contaminadas, mapa de calor.

04 USGS Earthquakes — Sismos

`earthquake.usgs.gov/fdsnws`

Sin auth | Tiempo real

BRONZE Sismos del último día/semana en GeoJSON: magnitud, profundidad, coordenadas, tsunami.

SILVER Parsear GeoJSON a tabular, clasificar por escala (leve/moderado/fuerte), asignar región por coordenadas.

GOLD `fact_sismos_diarios`, `dim_region`. Dashboard: mapa de sismos, distribución de magnitudes.

05 GitHub Events — Open Source

`api.github.com`

Sin auth (60 req/h) | Tiempo real

BRONZE Eventos públicos: PushEvent, PullRequestEvent, IssuesEvent, ForkEvent, etc.

SILVER Clasificar por tipo de evento, extraer lenguaje del repo, parsear actor, filtrar bots.

GOLD `fact_actividad_diaria`, `dim_lenguaje`, `dim_tipo_evento`. Dashboard: lenguajes más activos, ratio PRs/issues.

06 Citybikes — Bicicletas públicas

`api.citybik.es/v2`

Sin auth | Cada 2-5 min

BRONZE Snapshot de estaciones (EcoBici, Citi Bike...): bikes disponibles, slots vacíos, coordenadas, timestamp.

SILVER Enriquecer con barrio/zona por coordenadas, calcular ocupación, filtrar estaciones inactivas.

GOLD `fact_ocupacion_por_hora`, `dim_estacion`, `dim_zona`. Dashboard: estaciones más saturadas, uso por hora.

07 OpenSky — Tráfico aéreo

`opensky-network.org/api`

Sin auth (anónimo) | Cada 10 s

BRONZE Snapshot de aviones en vuelo: lat, lon, altitud, velocidad, rumbo, país de origen, indicativo.

SILVER Filtrar por región, clasificar fase de vuelo (ascenso/crucero/descenso por altitud), enriquecer con país/continente.

GOLD `fact_trafico_por_hora`, `dim_vuelo`, `dim_region`. Dashboard: mapa de vuelos activos, densidad por zona.

08 NASA FIRMS — Focos de incendio

`firms.modaps.eosdis.nasa.gov/api/area`

MAP_KEY gratis | Near real-time (~1 h)

BRONZE Detecciones de fuego activo (lat, lon, brillo, confianza, FRP, satélite, fecha) por bounding box de Argentina. Respuesta en CSV.

SILVER Filtrar por confianza, deduplicar detecciones entre satélites, clasificar intensidad por FRP/brillo, asignar provincia.

GOLD `fact_focos_diarios`, `dim_provincia`, `dim_satelite`. Dashboard: mapa de incendios activos, provincias más afectadas.

09 Transporte Buenos Aires

`api-transporte.buenosaires.gob.ar`

Registro gratis | Tiempo real (~30 s)

BRONZE Posiciones GPS de colectivos / subte vía GTFS-realtime (parsear el protobuf a tabular): línea, vehículo, lat/lon, velocidad, timestamp.

SILVER Filtrar por línea/zona, calcular velocidad y estado, enriquecer con el GTFS estático (nombre de línea, recorrido).

GOLD `fact_posiciones_por_hora`, `dim_linea`, `dim_franja_horaria`. Dashboard: mapa de unidades en vivo, horas pico.

⚠️ **Más setup**: registrarse para las credenciales y parsear GTFS-realtime (protobuf). Para quienes quieran un desafío extra.

10 Where the ISS at? — ISS

`api.wheretheiss.at/v1/satellites/25544`

Sin auth | Tiempo real

BRONZE Snapshot de la posición de la ISS (lat, lon, altitud, velocidad, visibilidad día/noche, timestamp) en cada ingesta.

SILVER Determinar si está sobre océano o tierra / qué país por coordenadas, fase día/noche, deltas de posición entre snapshots.

GOLD `fact_paso_por_region`, `dim_region`. Dashboard: traza orbital en el mapa, % de tiempo sobre tierra vs océano.

ℹ️ **La más simple en datos**: el valor está en acumular la trayectoria con el tiempo.

Todo lo demás, en GitHub (rama `main`, carpeta `TpFinal/`): el esqueleto sugerido de la carpeta del grupo, en README.md; el template del README del grupo, en grupos/G00/README.md; y las dudas de git, en git-guia.md.